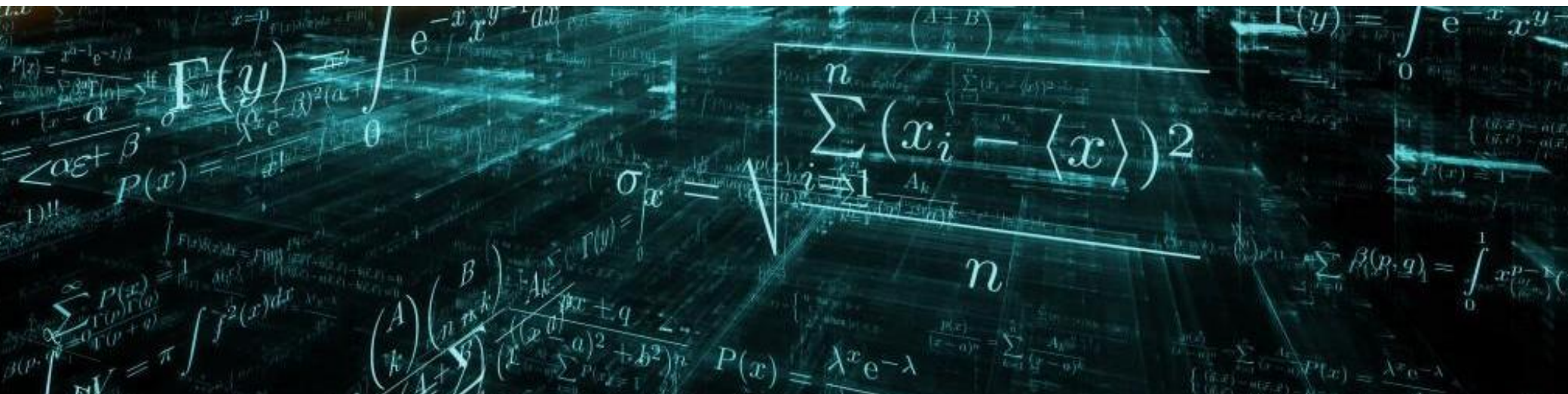


Multimedia Retrieval

Chapter 4: Index for Text Retrieval

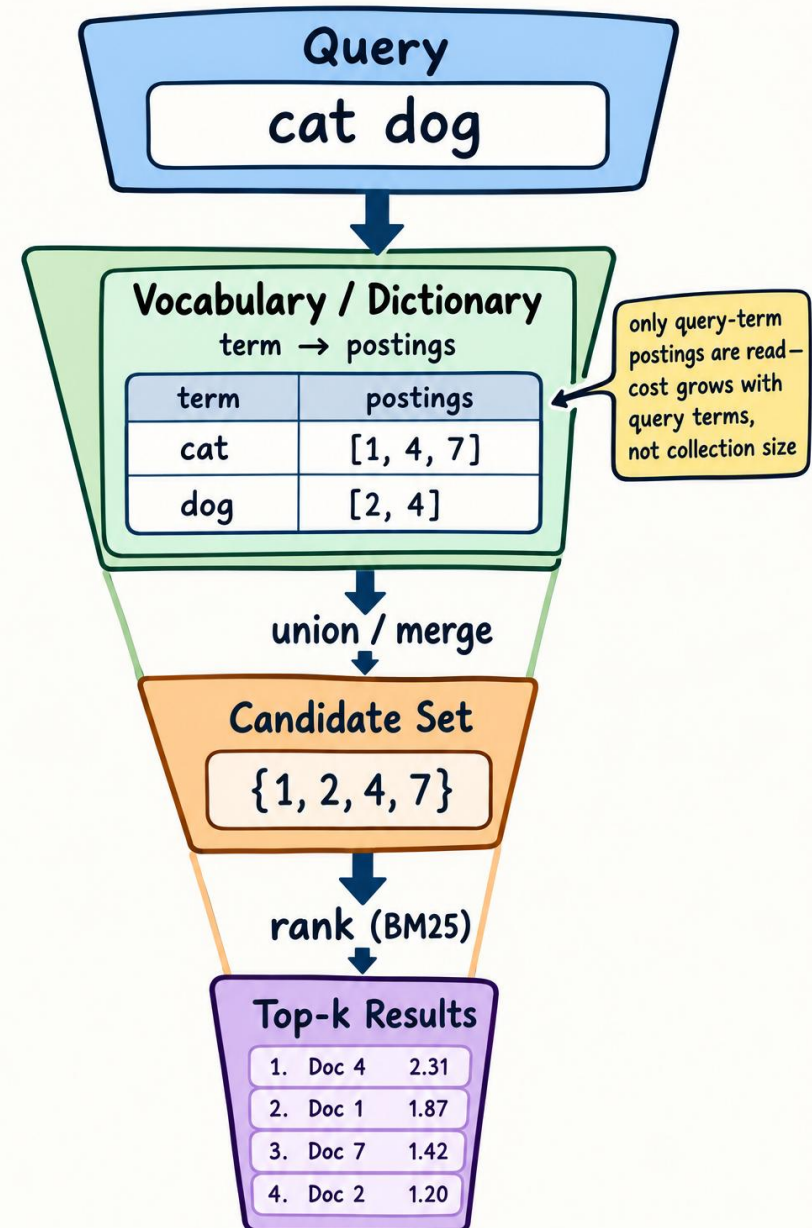
Dr. Roger Weber, roger.weber@gmail.com

- 4.1 Inverted Files
- 4.2 Ranked Retrieval
- 4.3 Filtering on Metadata
- 4.4 Practical Frameworks
- 4.5 Scaling Out



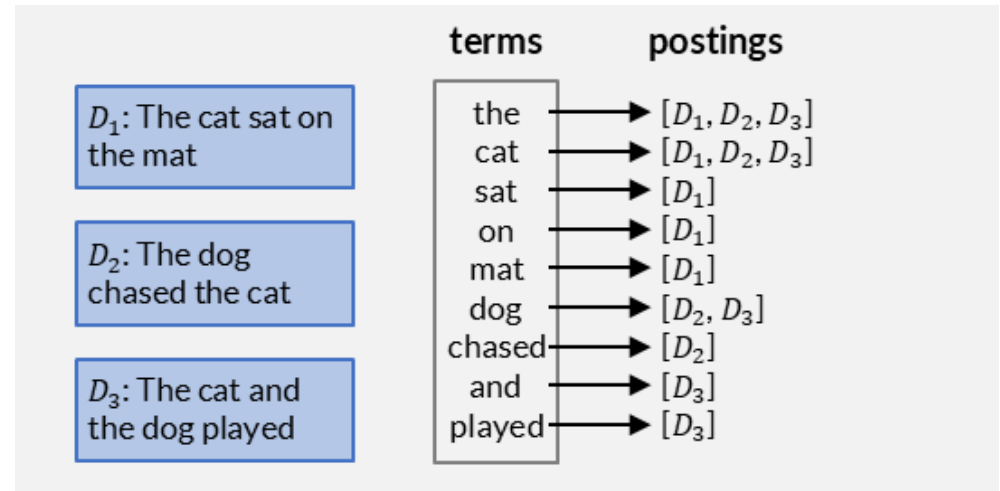
Making retrieval fast

- We can turn text into terms (ch1-3) and say what a good result is (ch2)
- Missing piece: find the matching documents quickly among millions
 - Scoring every document against every query, one by one, does not scale
 - Fast retrieval balances three goals: result quality, query latency, index storage
- **Inverted index:** store by term, mapping each term to the documents that contain it
 - Defining property: a query reads only its query-term postings, a small fraction of the collection, not every document
 - Has held this balance since the 1970s and still sits at the core of modern search
 - Dense vector methods only recently matched its speed, and pair with it rather than replace it



Inverted Files

- Observation: the term-document matrix is **sparse**
 - each document holds only tens to thousands of distinct terms
 - the vocabulary runs to millions of terms, so almost every cell is zero
- **Inverted index**: keep only the non-zeros, organised by term
 - each term points to its **postings list**: the IDs of the documents that contain it
 - document frequency comes for free: $df(t)$ = length of the postings list
- One structure serves every classical model (Boolean, vector space, BIR, BM25)
 - key assumption: only query terms affect relevance, so we read only their postings



Query read cost

- **Notation:** N documents, M vocabulary terms, K terms per document (K much smaller than M), L query terms (about 5)
 - A query reads only its query terms' postings, never the whole collection
 - an average postings list holds $N \cdot K / M$ entries
 - L terms read a fraction L/M of a full scan
 - with $L = 5$ and $M = 1,000,000$, that is five millionths
- Cost scales linearly with the collection
 - double the documents, double the read
 - table assumes 4 bytes per posting (a 32-bit document ID)

$$\text{entries read} \approx \frac{N \cdot K \cdot L}{M} = (N \cdot K) \cdot \frac{L}{M}$$

Collection	N	K	M	Full index	Read/query
E-book titles	30 M	6	1 M	~0.7 GB	~3.5 KB
Code search	50 M	8	500 K	~1.5 GB	~16 KB
Web pages	10 B	600	30 M	~21.8 TB	~3.8 MB

The structure of the index

- The index has three parts
 - **vocabulary** (dictionary): the M distinct terms, the lookup key
 - **postings list**: the document IDs for each term
 - **document table**: per-document metadata (title, author, URL), not searched term by term
- Postings **must** be kept sorted by increasing document ID
 - the merge-based algorithms on the next slides rely on it
 - out-of-order postings break AND / OR / AND-NOT as stream merges
- For Boolean over set-of-words, postings need only document IDs

ID	Title	Author	Year
1	Database Systems: The Complete Book	Garcia-Molina et al.	2008
2	Pattern Recognition and Machine Learning	Bishop	2006
4	Operating System Concepts	Silberschatz et al.	2018
50	Frankenstein	Mary Shelley	1818

Term	Postings (document IDs)
computer	[9, 10, 12, 14]
design	[8, 9]
software	[9, 13]
systems	[1, 14]
the	[1, 7, 8, 9, 13, 16, 19, 20, 26, 28, 31, 32, ...]

Boolean retrieval over postings

- Each postings list is a set of document IDs, so Boolean operators become set operations
 - AND: intersection
 - OR: union
- Bare NOT "term" is **not** supported
 - postings list the documents that **have** the term, not the ones that lack it
 - the complement can be almost the whole collection
 - NOT inside AND: "cat AND NOT dog" is a set subtraction (cat - dog), which we support
 - NOT inside OR: "cat OR NOT dog" has no bounded result, and little meaning as a query
- Set version is clear but loads whole lists; the stream merge (next) scales it down

```
cat   = index['cat']      # [4, 5, 12, 13, 14, 15, 20, 22, 30, 34]
dog   = index['dog']      # [1, 3, 4, 6, 9, 10, 13, 21, 22, 23, 29, 30]
horse = index['horse']    # [6, 10, 11, 14]
bird  = index['bird']     # [2, 3, 8, 15, 26, 35, 36]
```

```
cat & dog      # AND
# -> [4, 13, 22, 30]
```

```
horse | bird   # OR
# -> [2, 3, 6, 8, 10, 11, 14, 15, 26, 35, 36]
```

```
cat - dog      # AND NOT
# -> [5, 12, 14, 15, 20, 34]
```

```
(cat | dog) & (horse | bird)
# -> [3, 6, 10, 14, 15]
```

Merging sorted postings as streams

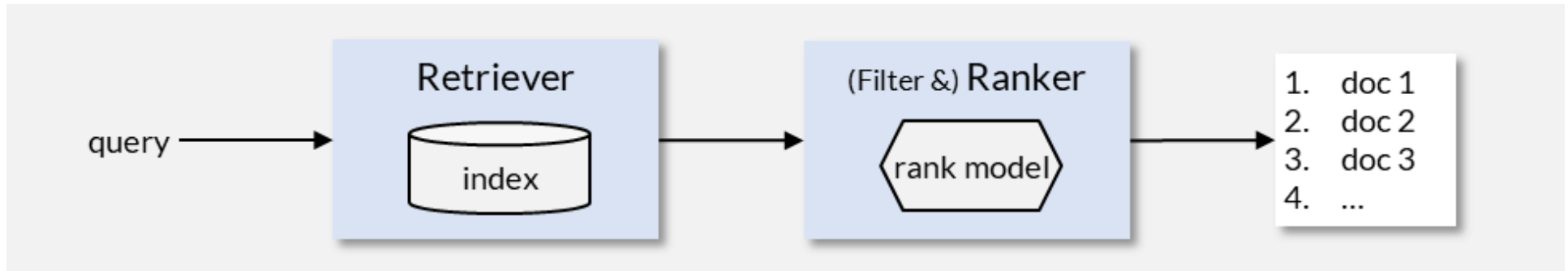
- Loading whole postings lists into memory does not scale to millions of entries
 - instead read them as **sorted streams**, advancing one entry at a time
- Merge rule per operator (heads = current entry of each stream)
 - **AND**: advance the smaller head; on a tie, emit and advance both
 - **OR**: emit the smaller head each step
 - **AND NOT**: emit a cat entry only when strictly smaller than the dog head
- Every operator consumes and emits sorted streams, so they **compose**
 - the trace on the right runs "cat AND dog" step by step

cat head	dog head	comparison	action	out
1	3	$1 < 3$	advance cat	
4	3	$4 > 3$	advance dog	
4	4	equal	emit, advance both	4
10	8	$10 > 8$	advance dog	
10	10	equal	emit, advance both	10
exhausted	12	cat empty	stop	

► [Demo: ch04-01-inverted-index.ipynb](#)

Ranked Retrieval over Inverted Files

- Boolean returns an unordered candidate set; ranking returns the best matches first
- Two stages over the same inverted index
 - **retriever**: gather candidates as the union of the query-term postings (a Boolean OR)
 - **ranker**: score the candidates and keep the top results
- A document with no query term scores zero under all three models (BIR, vector space, BM25)
 - an optional filter drops candidates on a metadata condition before ranking
 - what differs between models: the postings content and the scoring function



Term-at-a-time (TAAT)

- Process one query term fully before moving to the next
 - walk the term's postings, add its c_j weight to a per-document running score
 - after all terms, the score dictionary holds every candidate's final score
- Mirrors the Boolean set approach, but accumulates weights instead of intersecting
- Cost
 - the score dictionary grows large for common terms
 - no early pruning: scores are incomplete until the last term

```
def search_TAAT(query, k):
    query_vector = analyzer.set_of_words(query)
    # filter terms and obtain c_j-weights
    # returns a list of pairs (term_j, c_j)
    term_weights = query_weights(query_vector)

    scores = defaultdict(int)

    # iterate over terms and their postings
    for (term, weight) in term_weights:
        for doc_id in index[term]:
            # add this term's c_j to the running score
            scores[doc_id] += weight

    # avoid a full sort: use a heap for the top k
    topk = TopKList(k)
    for doc_id, score in scores.items():
        topk.add(doc_id, score)
    return topk
```

Document-at-a-time (DAAT)

- Extends the sorted-stream merge: advance all query-term streams in parallel
 - produce one candidate at a time, in document-ID order
 - each candidate gets its complete score at once, fed to a bounded top-k heap
- Memory bounded by the heap size k; no full score dictionary
- Can prune once the heap is full: skip candidates whose best possible score cannot enter the top k (WAND)
 - preferred in production; slightly more bookkeeping

```
def search_DAAT(query, k):
    query_vector = analyzer.set_of_words(query)
    term_weights = query_weights(query_vector)

    # one iterator per term, fetch the first posting
    iters = [iter(index[term]) for (term, _) in term_weights]
    nexts = [next(it, None) for it in iters]

    topk = TopKList(k)
    while not all(e is None for e in nexts):
        # smallest doc ID across all stream heads
        smallest = min(nexts, key=lambda x: x or math.inf)

        # sum c_j for every term whose head equals smallest
        score = 0
        for j in range(len(nexts)):
            if nexts[j] == smallest:
                score += term_weights[j][1]
        topk.add(smallest, score)

        # advance every stream whose head was smallest
        for i, e in enumerate(nexts):
            if e == smallest:
                nexts[i] = next(iters[i], None)
    return topk
```

What each model stores in the postings

- Same retriever and ranker; models differ only in **what postings store** and **how a candidate is scored**
 - BIR: postings need only document IDs
 - Vector space: add $\text{tf}(D, t)$ per posting; store each document's norm for cosine
- BM25 - what the index must carry
 - postings hold **(doc_id, tf)** tuples: the term frequency per document
 - **df** comes for free: the length of the postings list
 - **|D|** (document length): stored once per document
 - **avgdl**: a single index-wide value, need not be exact
 - **k, b**: constants supplied with the query, not stored

$$\text{sim}_{\text{BM25}}(Q, D) = \sum_j \text{idf}(t_j) \cdot \frac{\text{tf}(D, t_j) (k + 1)}{\text{tf}(D, t_j) + k \left(1 - b + b \frac{|D|}{\text{avgdl}} \right)}$$

$$\text{idf}(t_j) = \log \frac{N - \text{df}(t_j) + 0.5}{\text{df}(t_j) + 0.5}$$

► [Demo: ch04-02-ranked-retrieval.ipynb](#)

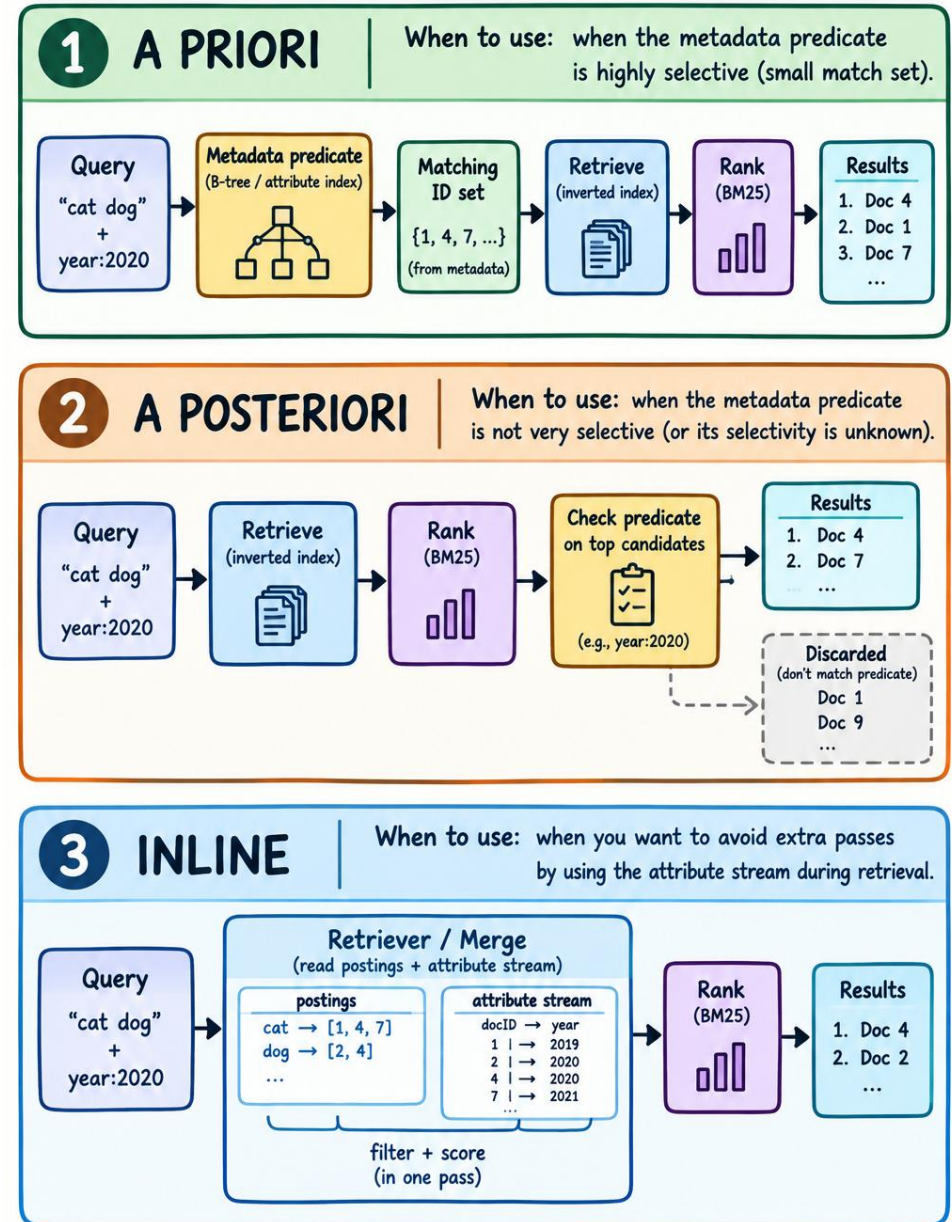
Filtering on metadata

- A **predicate** is a condition on metadata, e.g. `year > 2000` or `genre = "Computer Science"`

- the retrieval algorithm is unchanged: a failing candidate is simply not scored or returned

- Three strategies, differing in **when** the predicate is checked

- **a priori**: evaluate the predicate first (B-tree or inverted list on the attribute), pass the matching IDs into the retriever; best when the attribute is indexed
- **a posteriori**: rank first, check the predicate only on the documents about to be returned; when no index exists
- **inline**: store the attribute in a stream aligned with the postings, test it during the merge; filtering and scoring in one pass



Full-text search in PostgreSQL

- Documents already in a database? Search them in place: no separate service, no index to sync
 - full-text predicates combine with ordinary SQL: filter, join, transaction
- PostgreSQL building blocks
 - **tsvector**: sorted, normalized lexemes;
tsquery: Boolean/phrase; @@: match
 - a **GIN index** makes it a genuine inverted index (lexemes in a B-tree, compressed postings)
- **ts_rank** is not BM25
 - ranks by lexeme frequency (+ optional field weights); no corpus-wide idf
 - good for matching and filtering, only adequate for relevance ranking

```
-- 1. Define a table for movies
CREATE TABLE movies (
  id      SERIAL PRIMARY KEY,
  title   TEXT NOT NULL,
  overview TEXT NOT NULL,
  rating  REAL,
  year    INT,
  tsv     tsvector
);

-- 2. Populate the tsvector column for full-text search
--   'A' = highest weight (title gets more influence)
--   'B' = lower weight (overview contributes less)
UPDATE movies
SET tsv = setweight(to_tsvector(title), 'A') ||
        setweight(to_tsvector(overview), 'B');

-- Create a GIN index for fast lookup
CREATE INDEX movies_tsv_idx ON movies USING GIN(tsv);

-- 3. Search with ranking and a predicate on year
SELECT id, title, year,
       ts_rank(tsv, to_tsquery('Star & Wars')) AS r
FROM movies
WHERE tsv @@ to_tsquery('Star & Wars')
      AND year < 2000
ORDER BY r DESC
LIMIT 10;
```

SQLite FTS5: BM25 for free

- SQLite embeds the whole database in one file, no server: the lightest full-text option
 - common for on-device and application-embedded search
- **FTS5** stores a genuine inverted index behind a virtual table
 - **MATCH** takes the same Boolean/phrase syntax as a dedicated engine
 - ranks with true **BM25** via a built-in `bm25()` function
- Ranking lines up with the hand-rolled BM25 of the previous section

```
-- Virtual table backed by an FTS5 inverted index
CREATE VIRTUAL TABLE books
  USING fts5(title, author, tokenize = 'porter unicode61');

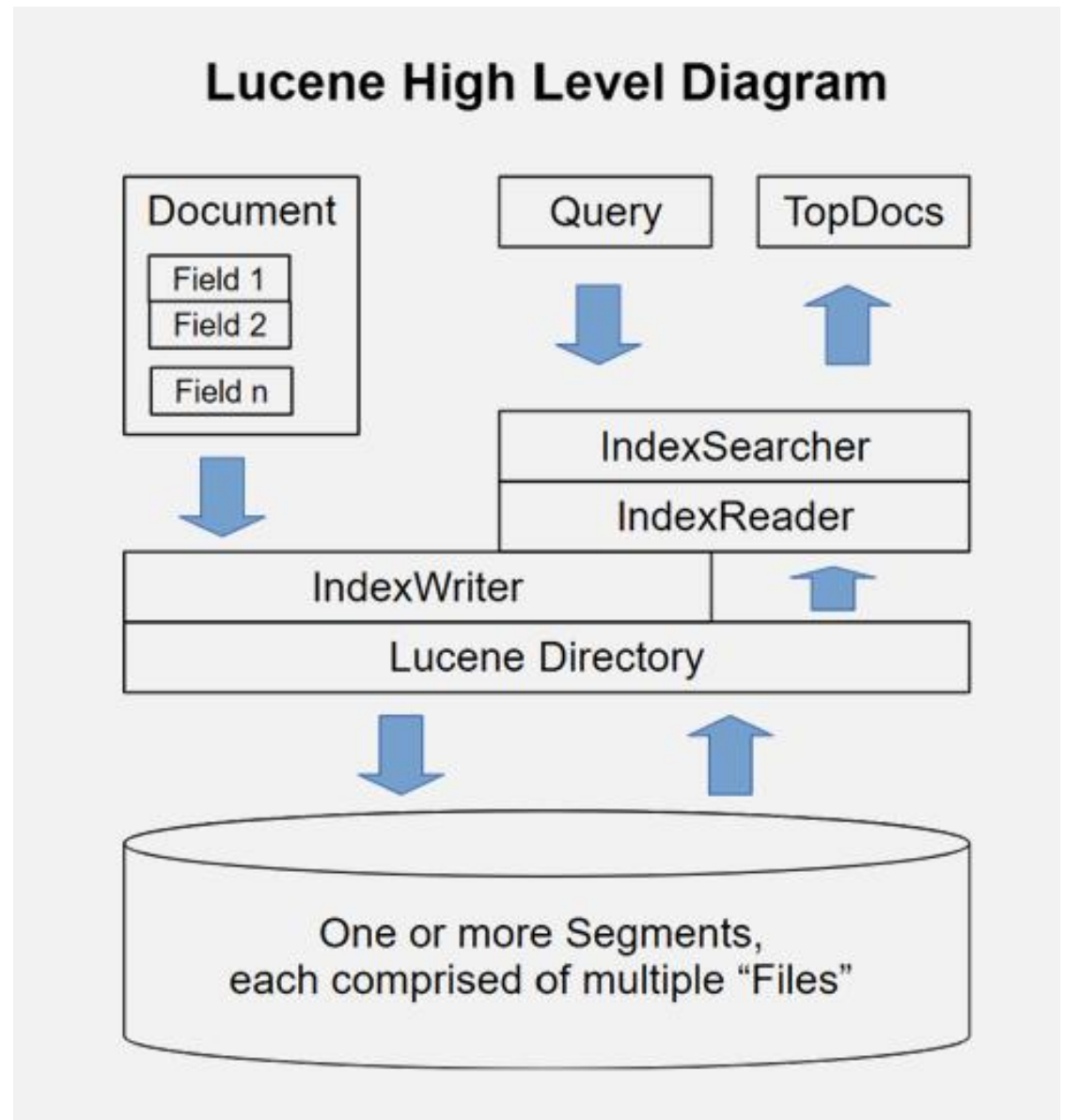
-- Load documents (one row per book)
INSERT INTO books (title, author)
  VALUES ('Database Systems', 'Garcia-Molina');

-- Ranked search: MATCH selects candidates, bm25() scores them
SELECT title, bm25(books) AS score
FROM books
WHERE books MATCH 'database systems'
ORDER BY score;  -- bm25() is negative: ascending = best first
```

► **Demo:** [ch04-03-database-search.ipynb](#)

Lucene

- Apache Lucene (Doug Cutting, 1999): the open-source Java library behind most production search
 - the full toolkit: analysis, inverted indexing, term weighting, ranking
 - current major version Lucene 10; concepts stable across versions
- Two paths meet at the **Directory** storage abstraction
 - write: Document -> IndexWriter -> Directory -> Segments
 - read: Query -> IndexSearcher -> IndexReader -> Directory -> Segments -> TopDocs



Documents, fields, and segments

- A document is a set of named **fields**; a field's type sets three independent choices: tokenized? indexed? stored?
 - fields are indexed independently: title:house vs body:house are distinct terms
 - the **analyzer** tokenizes; identical for indexing and querying (else rebuild)
- The index is a set of immutable **segments**, one per IndexWriter flush
 - no primary key: a docId can change on merge, so deletes/updates identify by content
 - delete-by-term flips a live-bit; update = delete-by-term + add
 - a merge policy drops deleted docs; search runs over all live segments in parallel

```
// Comparing analyzer output for the same text
var text = "I think text's values' color goes here; WHAT happens ...";

// StandardAnalyzer: lowercase, strip punctuation, keep possessives
print_tokens(new StandardAnalyzer(), text);
// -> i think text's values color goes here what happens ...

// EnglishAnalyzer: + stop-word removal + Porter stemming
print_tokens(new EnglishAnalyzer(), text);
// -> think text valu color goe here what happen ...

// Custom stop words
var stopWords = new CharArraySet(Arrays.asList("i", "do"), false);
print_tokens(new EnglishAnalyzer(stopWords), text);
// -> think text valu color goe here what happen ...
```

```
// Set up analyzer, directory, and writer
Analyzer analyzer = new EnglishAnalyzer();
Directory directory = FSDirectory.open(Paths.get("./index"));
IndexWriter writer = new IndexWriter(directory,
                                     new IndexWriterConfig(analyzer));

// Build a document from application data
Document doc = new Document();
doc.add(new TextField("title", "Star Wars", Field.Store.YES));
doc.add(new IntField("year", 1977, Field.Store.YES));
doc.add(new TextField("body", "A long time ago ...", Field.Store.NO));
writer.addDocument(doc);

// Close the writer to flush a new segment
writer.close();
```

Scoring function

- Lucene ranks with **BM25 by default**, computed per field (df and length use that field's stats)
 - each clause contributes a per-term score_i ; the document score is their sum
 - filter clauses contribute zero
- Two practical additions to the ch1 BM25
 - a per-clause **boost** factor
 - the **$\log(1 + \dots)$** idf variant, positive even for very common terms
- Optimization: drop the constant $(k_1 + 1)$ numerator factor
 - order unchanged; Lucene's raw scores come out roughly half

$$\text{score}(D, Q) = \sum_i \text{score}_i$$

$$\text{score}_i = \text{boost}_i \cdot \text{idf}_i \cdot \frac{\text{tf}_i (k_1 + 1)}{\text{tf}_i + k_1 \left(1 - b + b \frac{dl}{\text{avgdl}}\right)}$$

$$\text{idf}_i = \log \left(1 + \frac{N - n_i + 0.5}{n_i + 0.5} \right)$$

Querying

- An **IndexSearcher** over the directory; a query parser applies the same analyzer used at indexing
 - results come back as **TopDocs**: the best-scoring documents and their scores
- Rich query types
 - field-scoped terms with boosts, ranges, wildcards, fuzzy matching
 - clauses: **MUST**, **SHOULD**, **MUST_NOT**, **FILTER** (filter matches but does not score)
- Queries can be built programmatically (below): a **FILTER** on year, a **MUST** title term, a boosted **SHOULD** body term

```
// Set up a searcher and a multi-field query parser
IndexSearcher searcher = new IndexSearcher(
    DirectoryReader.open(directory));

QueryParser parser = new MultiFieldQueryParser(
    new String[]{"title", "body"}, analyzer);

// Simple keyword query (searches title and body, ranked by BM25)
Query q = parser.parse("star wars");
TopDocs results = searcher.search(q, 10);

// Access stored fields (Lucene 10 API)
StoredFields storedFields = searcher.getIndexReader().storedFields();
for (ScoreDoc hit : results.scoreDocs) {
    Document d = storedFields.document(hit.doc);
    System.out.printf("%.2f %s (%s)\n", hit.score,
        d.get("title"), d.get("year"));
}

// Programmatic query with filter, boost, and explain
Query query = new BooleanQuery.Builder()
    .add(IntField.newRangeQuery("year", 1950, 2000), Occur.FILTER)
    .add(new TermQuery(new Term("title", "shawshank")), Occur.MUST)
    .add(new BoostQuery(
        new TermQuery(new Term("body", "hope")), 1.5f), Occur.SHOULD)
    .build();

TopDocs top = searcher.search(query, 10);
// Score breakdown for the top hit
System.out.println(searcher.explain(query, top.scoreDocs[0].doc));
```

Explain: the score breakdown

- **explain** decomposes a document's score into each clause's contribution
 - every BM25 quantity is visible: idf ($\log(1 + \dots)$ variant), saturating tf with k_1 , b , dl , $avgdl$, and the boost
 - filter clauses contribute zero but must be satisfied
- Invaluable for diagnosing why a document ranks where it does, and for tuning field boosts

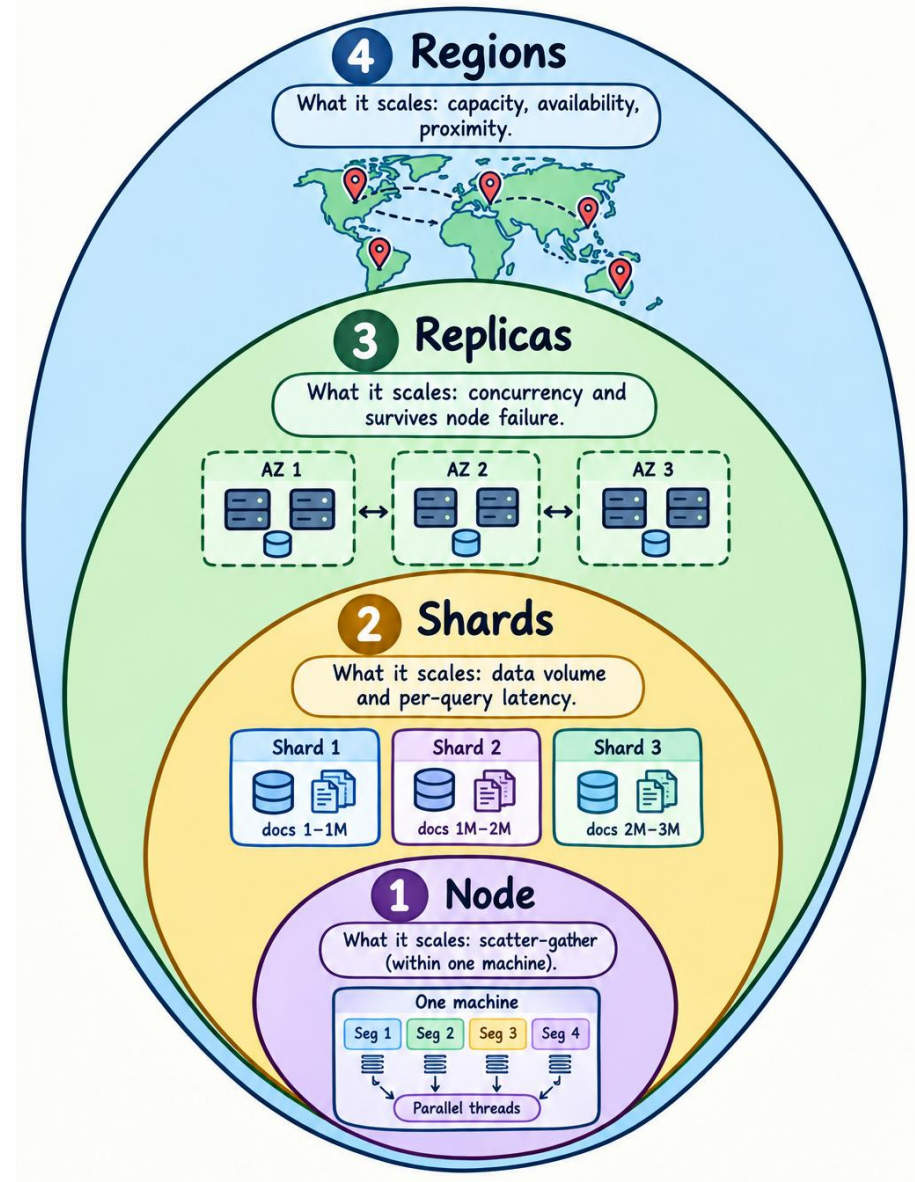
```
System.out.println(  
    searcher.explain(query, top.scoreDocs[0].doc));
```

```
7.4038 = sum of:  
0.0 = match on required clause, product of:  
0.0 = # clause  
1.0 = year:[1950 TO 2000]           // FILTER: must match, no score  
1.0 = year:[1990 TO 2000]           // SHOULD: contributes 1.0  
3.0980 = weight(title:shawshank in 0) [BM25Similarity], result of:  
3.0980 = score(freq=1.0), computed as boost * idf * tf from:  
6.5023 = idf, computed as  $\log(1 + (N - n + 0.5) / (n + 0.5))$  from:  
1 = n, number of documents containing term  
999 = N, total number of documents with field  
0.4765 = tf, computed as  $\text{freq} / (\text{freq} + k_1 * (1 - b + b * dl / avgdl))$   
1.0 = freq, occurrences of term within document  
1.2 =  $k_1$ , term saturation parameter  
0.75 =  $b$ , length normalization parameter  
2.0 =  $dl$ , length of field  
2.2533 =  $avgdl$ , average length of field  
3.3057 = weight(body:hope in 0) [BM25Similarity], result of:  
3.3057 = score(freq=1.0), computed as boost * idf * tf from:  
1.5 = boost                        // the BoostQuery factor  
4.7687 = idf, computed as  $\log(1 + (N - n + 0.5) / (n + 0.5))$  from:  
8 = n, number of documents containing term  
1000 = N, total number of documents with field  
0.4621 = tf, computed as  $\text{freq} / (\text{freq} + k_1 * (1 - b + b * dl / avgdl))$   
1.0 = freq, occurrences of term within document  
1.2 =  $k_1$ , term saturation parameter  
0.75 =  $b$ , length normalization parameter  
8.0 =  $dl$ , length of field  
8.335 =  $avgdl$ , average length of field
```

► [Demo: ch04-04-lucene.ipynb](#)

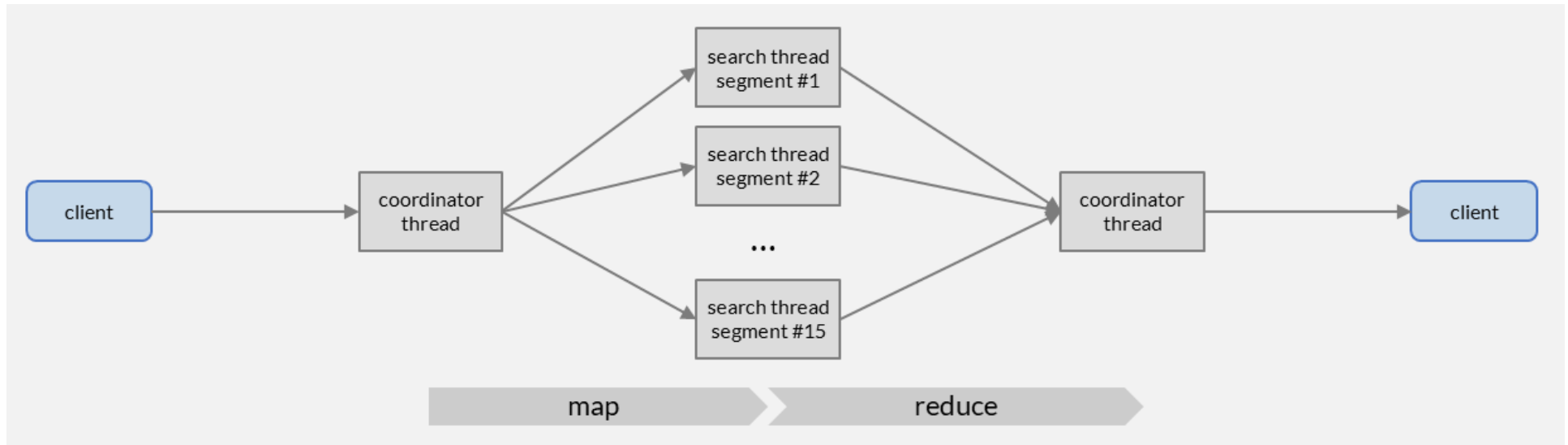
Scaling Out

- One Lucene index handles up to ~2.1B documents and parallelizes across segments
 - index > ~20 GB, latency is memory/disk bound, and one machine cannot serve hundreds of concurrent queries
- Four levels of parallelism:
 - nodes -> shards -> replicas -> regions
- Three engines built on Lucene
 - **Solr**: schema-driven, strong faceting and multilingual
 - **Elasticsearch**: schema-flexible REST API, log/security analytics (ELK)
 - **OpenSearch**: 2021 Apache 2.0 fork; ES and OpenSearch also do dense-vector (ANN) search



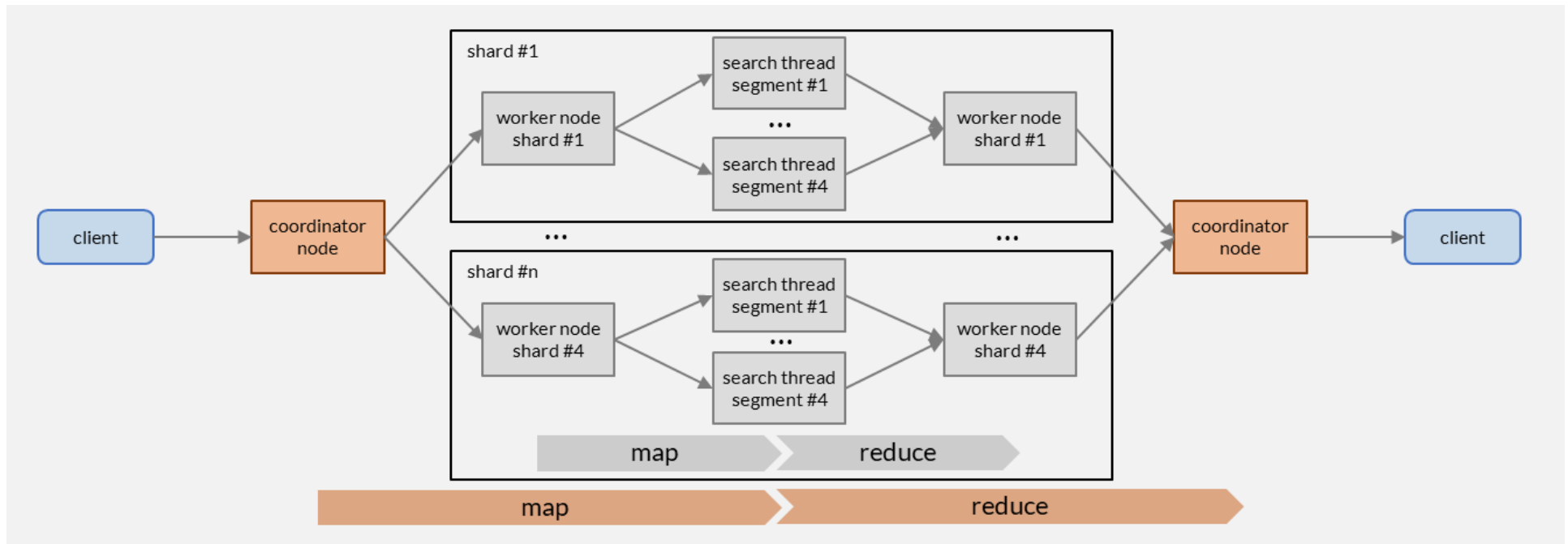
Parallelism within a node

- Scale on one machine: split the index into **segments** (a merge policy controls how many)
 - **scatter-gather** (map-reduce): a coordinator parses the query, hands each segment to a searcher thread, then merges the partial results
- Bounded speedup: with 99% of work in segment search, Amdahl's law caps it at ~13x over one thread
- Limits: latency still grows with index size; one query at a time runs at full width; a single server has no high availability



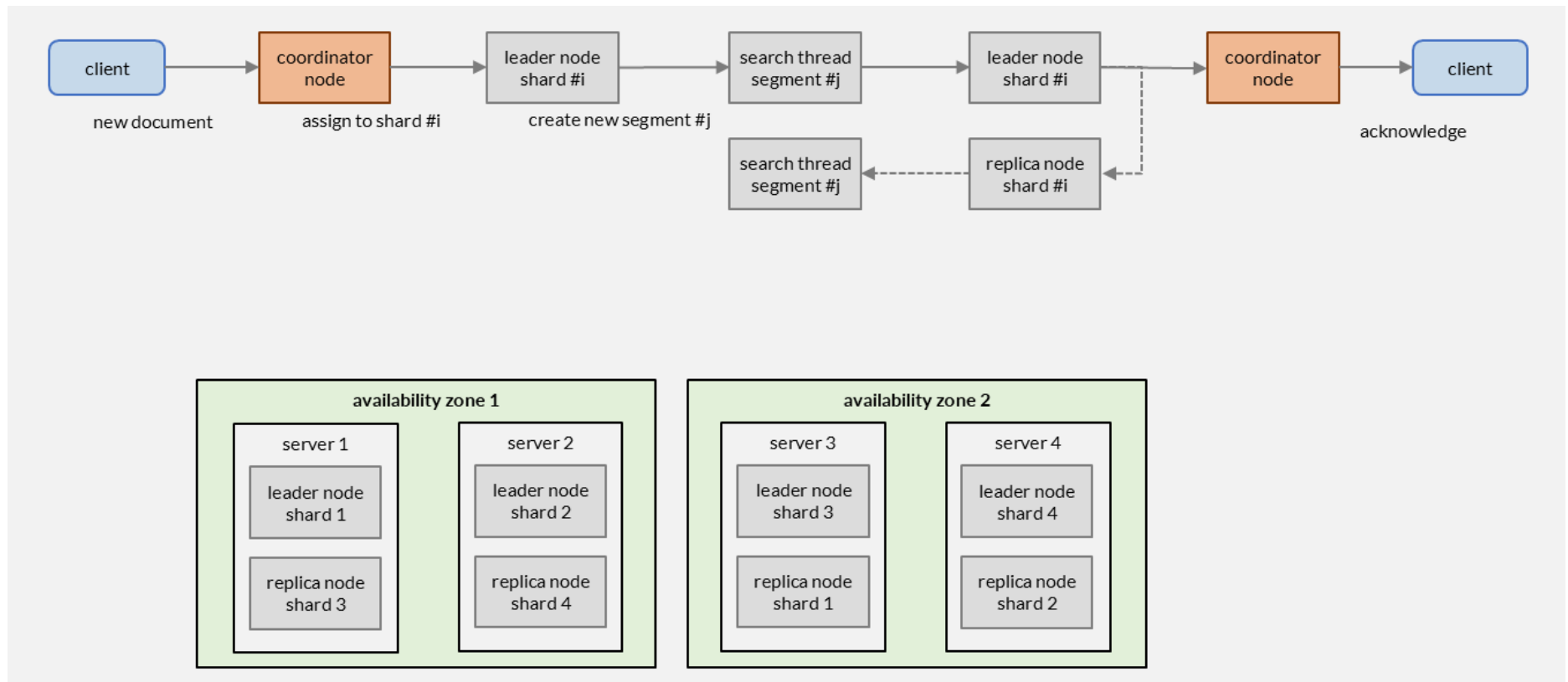
Sharding across nodes

- A **shard** is a distinct subset of documents, its own Lucene index (own segments and term statistics)
 - search becomes two-level map-reduce: scatter across shards, then across each shard's segments, reduce back up
 - per-shard term statistics mean the same document can score slightly differently by shard (acceptable)
- A coordinator assigns new documents to a shard (round-robin, or hashing an ID prefix to co-locate related docs)
- n shards handle up to n times the single-node limit; shard count is usually fixed (redistribution is expensive)



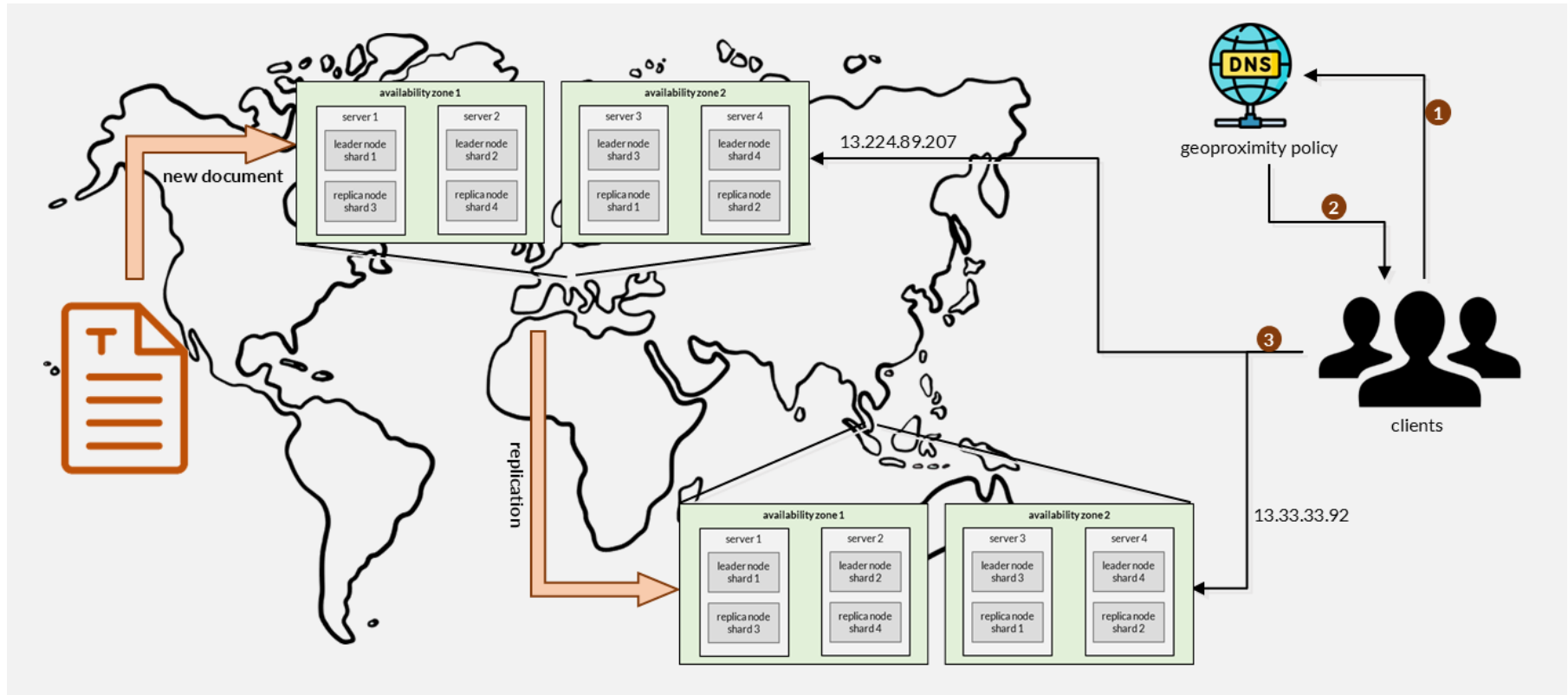
Replication and availability

- Sharding spreads data but does not survive node failure: **replicate each shard**
 - place replicas in different availability zones (not the same server, rack, or data center)
- Within a shard, the **leader** indexes and propagates to replicas; every replica returns identical results
 - each leader can also coordinate, routing a shard's part of a query to any replica or leader
- Adding replicas adds concurrent-query capacity roughly in proportion to their number



Distributing across regions

- Highest level: whole **clusters** in several geographic regions, near their users
 - writes go to a primary region and replicate across regions
 - DNS geoproximity resolves to the nearest region, with another as fallback
- A single query never splits across regions: 100-200 ms inter-region latency costs more than it saves
- Buys total capacity, higher availability, and lower latency for nearby clients, not lower per-query latency



Key takeaways

- Inverted index:
 - per-query cost grows with query length L , not collection size (reads a fraction L/M of a full scan)
- Postings stay sorted:
 - the retriever merges query-term postings, the ranker scores the candidates
- One structure serves
 - Boolean, BIR, vector space, and BM25;
 - only postings content and scoring differ
- DAAT: bounded memory and pruning;
TAAT: simpler, but a large score table and no pruning
- BM25 over postings
 - needs (doc_id, tf) tuples,
 - df (= postings length), $|D|$, and
 - an index-wide $avgdl$
- Filtering plugs in three ways:
 - a priori, a posteriori, inline
- Frameworks:
 - in-database FTS (PostgreSQL, SQLite),
 - the Lucene library,
 - distributed engines (Solr, ES, OpenSearch)
- Scale out in tiers:
 - node segments
 - shards (data, latency)
 - replicas (concurrency, availability)
 - regions (proximity)

Further reading

- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval**. *Cambridge University Press*. [Free HTML](#). The chapters on index construction and index compression cover this chapter's core structures in depth.
- Zobel, J., & Moffat, A. (2006). **Inverted Files for Text Search Engines**. *ACM Computing Surveys*, 38(2). The canonical survey of inverted-file representation, construction, and query evaluation, including compression and query-time optimizations.
- Robertson, S., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond**. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. [PDF](#). Derives the BM25 ranker used throughout the chapter, including the multi-field BM25F variant behind Lucene's per-field scoring.
- Büttcher, S., Clarke, C. L. A., & Cormack, G. V. (2010). **Information Retrieval: Implementing and Evaluating Search Engines**. *MIT Press*. An implementation-focused treatment of indexing, compression, and query processing.
- Apache Lucene: lucene.apache.org
- Apache Solr: solr.apache.org
- Elasticsearch: elastic.co
- OpenSearch: opensearch.org
- PostgreSQL Full Text Search: postgresql.org
- Zhou, Z. (2019). **Lucene IndexWriter: An In-Depth Introduction**. *Alibaba Cloud Blog*. [Article](#). Walks through IndexWriter internals: the concurrency model, segment creation, delete-by-term, flush, commit, and merge lifecycle.